

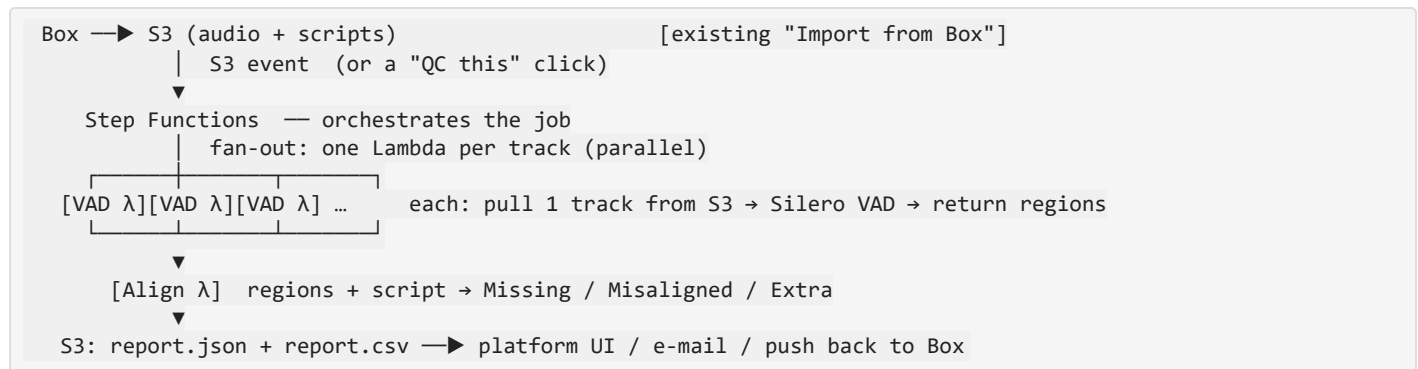
# Dialogue QC — Cloud Hosting Plan (Serverless)

**Goal:** run Dialogue QC as a service inside the Rian workflow — automatically, on cloud, pay-per-use — instead of a manual desktop app.

## Why it fits serverless

The audio is **huge**, but each VAD result is **tiny** (a few KB of "speech from X→Y"). So we fan work out **one function per track**, and move only the small results around. That's exactly what Lambda is good at.

## Architecture (AWS)



**S3** = audio/scripts/reports · **API Gateway** = async job API · **Lambda** (container image) = the existing Python core (parse · Silero VAD · align) · **Step Functions** = orchestrates the fan-out · **Frontend** = static site or an iframe in the platform.

**Fan-out is the win:** 12 tracks processed at once → **~6 s instead of ~75 s**, and it scales to any number of tracks.

## Lambda vs SageMaker

- **Lambda does all of it.** Silero VAD is tiny (2.3 MB, CPU-only) — no GPU, no SageMaker needed.
- **SageMaker is a later "if":** only if we add heavy ML (ASR to catch *wrong* lines, or voiceprint speaker-ID), which want GPU. Not required today.

## Logistics / what changes

- **Async, not synchronous** — API Gateway times out at 29 s; a job runs longer. `POST /qc` returns a **job ID**, the UI **polls status** (same progress pattern we have), then reads the report from S3.
- **Files come from S3/Box, not local disk** — a small change to the backend's file source (and the "browser can't read local paths" problem disappears).
- **Container-image Lambda** — deps (onnxruntime + model) exceed the 250 MB zip limit; container images (up to 10 GB) are the standard fix.
- **Large audio** — each Lambda pulls only **one** track, so it never holds all the audio → sidesteps the 10 GB `/tmp` limit. Client audio in S3 = encryption + IAM.

## Rian integration & cost

QC becomes a **pipeline step, not a separate app**: tracks landing in S3 via **Import-from-Box** fire an S3 event that runs the QC Step Function **automatically**; the report is surfaced in the platform UI, pushed back via **Export-to-Box**, and/or emailed — QC on ingest, no manual step. **Cost is pay-per-use: cents per episode, ~zero when idle.** The **Python core ports over almost unchanged**; the desktop app stays as the offline option. New work = *containerize · swap disk→S3 · wrap in Step Functions + API Gateway*.